函数指针不是炫技：从 qsort 到回调状态机

嵌入式 Linux 手记　2026-10-14　公众号文章 · 原链接 mp.weixin.qq.com

想象一下没有函数指针的标准库：排序得按类型和规则各写一个，`qsort_int_asc`、`qsort_int_desc`、`qsort_str_len`……类型数乘以规则数，组合爆炸，而且每加一种就得改库源码。真实的 `qsort` 只有一个，靠最后一个参数包打天下：

```
qsort(arr,   n, sizeof(int),    cmp_asc);    // int 升序
qsort(arr,   n, sizeof(int),    cmp_desc);   // int 降序
qsort(names, n, sizeof(char *), cmp_len);    // 字符串按长度
```

三行调用，`qsort` 内部一个字节都没变，变的只有最后一个函数指针。这篇从这一个参数出发，拆到驱动工程师每天注册的那张表。

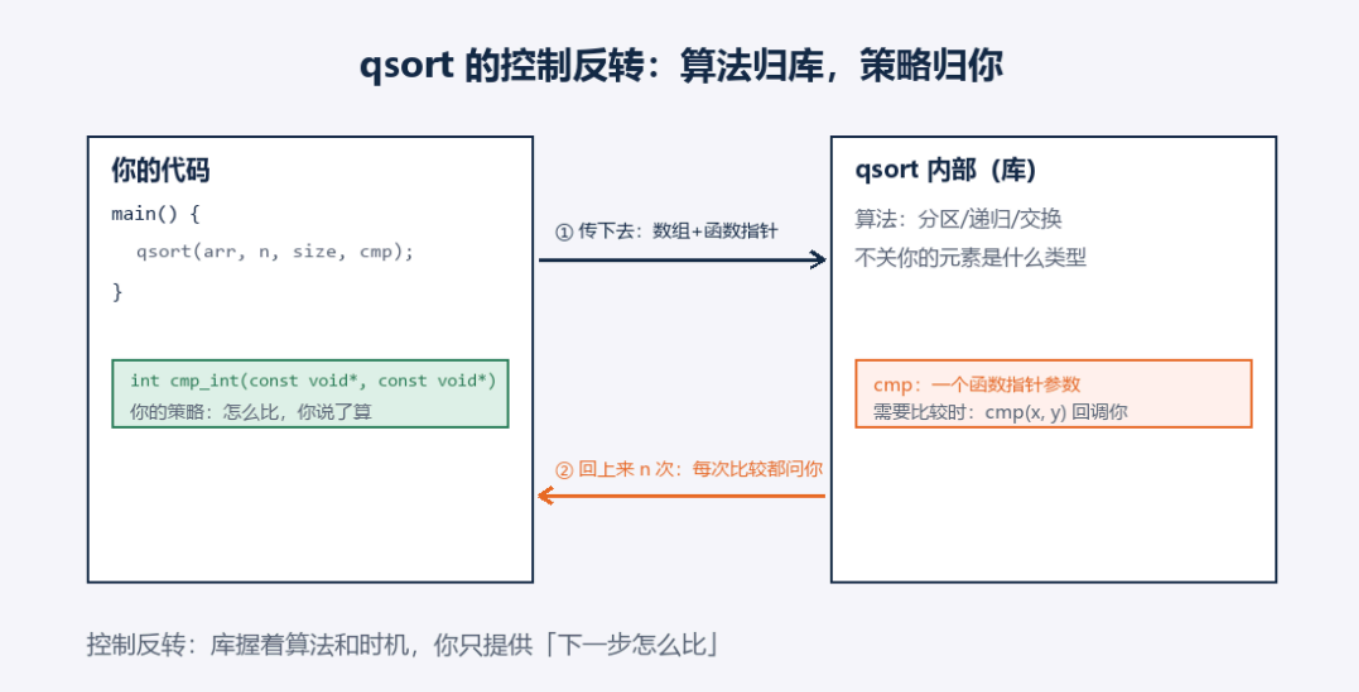
一、回调：控制权的反转

先看比较函数本尊：

```
int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);    // 三态：1 / 0 / -1
}
```

两个细节藏着 `qsort` 的设计思想。第一，参数是 `const void *`：`qsort` 不知道你会排什么类型，它只掌握两件事，"每块 n 字节"和"两个块的首地址"。类型知识全部留在你这边，比较函数里强转回真实类型才能解引用。第二，返回值用 `(x > y) - (x < y)` 而不是 `x - y`：当 `x` 是 `INT_MIN`、`y` 是 `INT_MAX` 时，减法回绕可能给出正号，排序结果错乱且极难复现；三态写法恒定输出 1、0、-1，这是比较函数的必修写法。

控制流的走向也值得注意。平时是"你调库"；`qsort` 是"你把函数交给库，库在排序进行到需要比较时回头调用你"。这叫控制反转：算法归库，策略归你，函数指针是 C 语言里传递策略的唯一通道。这个模式的另一个名字叫依赖倒置：高层定接口，低层填实现。C++ 的虚函数、Java 的接口，编译器在底层替你做的也就是这张函数表，函数指针是这个机制在 C 里的素颜形态。你的代码调下去一次，库的代码回上来成百上千次。



图：qsort 的控制反转：你的代码调下去一次，库的代码回上来 n 次

二、运行时真相：函数名也是地址

篇 1 讲过数组名在表达式里退化为地址；函数名是同一族现象。C11 §6.3.2.1 规定：函数指名符（function designator）在除了 `sizeof` 和取 `&` 之外的场合，自动转换为函数指针。所以 `qsort(arr, n, size, cmp_int)` 里的 `cmp_int` 传的已经是地址，`&cmp_int` 数值相同；调用 `table[cmd](fd)` 与 `(*table[cmd])(fd)` 也完全等价。

用四要素拆表项 `table[cmd]`，系列的工具最后一次收拢：

要素	答案	决定什么
指针的类型	`void (*)(int)`	能赋值给谁、表怎么声明
所指向的类型	`void(int)`，一个函数	调用形态
指针的值	代码区里该函数的入口地址	间接调用跳到哪
自身大小	8 字节	与普通指针一致

机器层面，间接调用与直接调用的差别一目了然：arm64 上直接调用编译成 `bl 目标`（地址在编译期焊死），间接调用是 `blr x0`（目标在寄存器里，运行期才知道）。把函数地址填进表的那

一刻，跳转目标才确定，这就是"运行期才决定行为"的底层含义。

一条工程惯例随之而来：回调必须判 `NULL` 。表里未注册的槽位默认就是 `NULL` ，不判空调用它，等于跳向地址 0，篇 3 的段错误换了个入口再来一次。顺带解决一个高频疑问： `sizeof` 能不能量一个函数？不能， `sizeof` 作用于函数名是约束违反；能量的是函数指针，得 8。面试里常用它区分"函数"与"函数指针"两个概念。

三、从一针到一张表：状态机与驱动注册

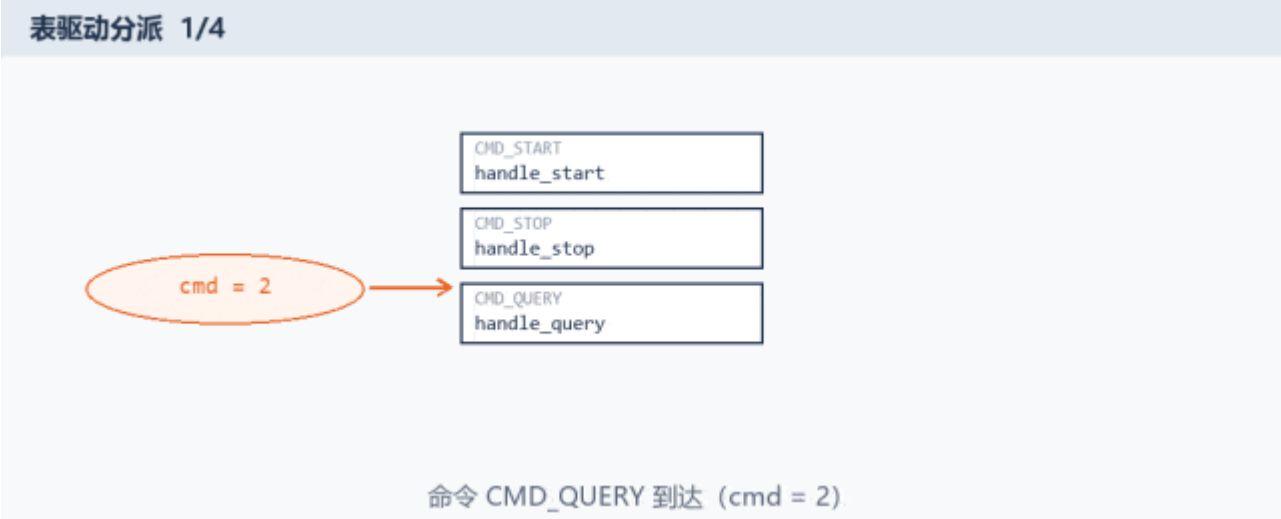
单个函数指针是"针"，排成一排就是"表"。先看命令分派：

```
typedef void(*Handler)(int fd);

staticconst Handler cmd_table[] = {
    [CMD_START] = handle_start,
    [CMD_STOP]  = handle_stop,
    [CMD_QUERY] = handle_query,     /* 未注册的槽位自动为 NULL */
};

voiddispatch(int cmd, int fd)
{
    if (cmd >= 0 && (size_t)cmd < sizeof cmd_table / sizeof cmd_table[0]
        && cmd_table[cmd] != NULL)
        cmd_table[cmd](fd);         /* 间接调用：框架回头调用你的代码 */
}
```

和 `switch` 版本比一比：新增一种命令， `switch` 要打开分发函数本体加一个 case，表驱动只加一行初始化和一个函数，分发逻辑一行不动。这就是"对扩展开放、对修改关闭"在 C 语言里的样子，一张 `const` 表就够。状态机同理：把 `{当前状态, 事件, 动作函数, 下一状态}` 排成转移表， `switch` 嵌套变成一次查表，协议解析和设备状态机都是这个写法。



图：表驱动分派动画：一条命令从到达、查表到间接调用的完整路径，以及与 switch 的扩展成本对比

这张表的终极形态在内核里。Linux VFS 的 `file_operations` 是驱动工程师每天都要填的函数指针表：

```
/* Linux VFS 的函数指针表（简化） */
struct file_operations {
    int     (*open)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char *, size_t, loff_t *);
};

static const struct file_operations my_fops = {
    .open = my_open,
    .read = my_read,        /* 内核在正确的时机回调这里 */
};
```

你注册一张表，内核在用户 open、read、write 的时机逐个回调。所谓写驱动，一大半工作就是填这张表。顺带留意它的初始化风格：`.open = my_open` 这种指定初始化器是 C99 的标准语法，成员顺序随意、可以缺省，缺省的成员自动为 `NULL`，与"回调判空"的惯例严丝合缝。

还剩一个问题：回调发生时，内核怎么把"你的数据"交还给你？回调签名是统一的，私有数据各不相同。内核的答案是 `container_of`：把你的数据结构埋在宿主结构体里，从回调拿到的成员地址，减去它在宿主里的偏移，一步反推出宿主指针。

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

工作队列、定时器回调全是这个模式：签名固定、数据随宿主走。到这里，函数指针的拼图集齐了：一个指针做回调，一排指针做分派，一张结构体表做框架，一个宏把数据找回来。

写在最后

把这篇压成五条：

1. 函数名在表达式里自动转为函数指针，`f` 与 `&f` 等价；这与数组退化同族，依据 C11 §6.3.2.1。
2. 比较函数写三态 `(x > y) - (x < y)`，不用 `x - y`：减法在 `INT_MIN` 与 `INT_MAX` 相邻时回绕，排序悄悄出错。
3. 新增命令加一行表、不动分发逻辑，这是开闭原则的 C 实现；`switch` 版本每加一条都要改本体。
4. 回调判 NULL 是工程铁律：未注册槽位就是 NULL，空调用等于跳地址 0。
5. 间接调用比直接调用多一次取址，极热路径上 profiling 说了算；但解耦收益通常远大于这点开销，内核满屏函数指针表就是证词。

下一篇是前七篇的收束：`char *s = "hello"` 之后 `*s = 'H'` 为什么会崩？字符串常量住在内存的哪个区，"只读"两个字如何从编译期一路管到运行期。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。